

Applications of Artificial Neural Network

HW1 - Experimental Report

羅政昕 R37141329

1. Problem Definition and Motivation

I was searching for a bit more meaningful classification problem that could be helpful to society if solved. After some time, I bumped into more societal and medical datasets like burnouts and addictions. Originally, I would like to apply Perceptron to work burnouts that are related to AI, but the datasets seem to be a bit too complicated to single-layered Perceptron. Later on, I found a mobile phone addiction dataset that has sensible and adequate features, so I decided to apply Perceptron on this dataset, and hopefully to create a working model that could help medical personnel or psychologists to judge if a mobile user is likely to become addicted to mobile phone usage, allowing early intervention to be done.

This problem (dataset) is suitable for a single-layer Perceptron, because its target is binary (addicted / not addicted), which is exactly what a single-layer Perceptron does best.

2. Dataset Description

2-1. Source:

The dataset is found on Kaggle (Link: <https://www.kaggle.com/datasets/jayjoshi37/smartphone-usage-and-addiction-prediction>). It is a synthetic dataset according to the description.

2-2. Features:

transaction_id	Only the data entry ID, omitted in data preprocessing.
user_id	Only the user ID, omitted in data preprocessing.
age	Age of the user

gender	Gender of the user
daily_screen_time_hours	Average daily phone screen time (hours) for the user.
social_media_hours	Average social media usage time (hours) on the phone for the user.
gaming_hours	Average gaming time (hours) on the phone for the user.
work_study_hours	Average work or study time (hours) on the phone for the user.
sleep_hours	Average sleeping time (hours) for the user.
notifications_per_day	Total number of phone notifications that the user receives per day.
app_opens_per_day	Number of times applications are opened per day.
weekend_screen_time	Average smartphone screen time during weekends.
stress_level	Self-reported stress category (Low / Medium / High).
academic_work_impact	Indicates whether smartphone usage impacts productivity (Yes / No).
addiction_level	Categorical addiction severity (None / Mild / Moderate / Severe). Omitted in pre-processing.
addicted_label	Binary classification target variable. (0 = Not Addicted, 1 = Addicted)

2-3. Labels:

The target classes for this dataset are 0 (Not Addicted) and 1 (Addicted)

3. Implementation

3-1. Data preprocessing:

For the smartphone addiction dataset, data preprocessing was necessary before training the Perceptron because the model can only operate on numeric input features.

First, the dataset was loaded from the CSV file and divided into input features X and the target label y, where “addicted_label” was used as the binary classification target. Some columns such as “transaction_id” and “user_id” were excluded because they are only identifiers and do not contain meaningful behavioral information for classification. The column “addiction_level” was also excluded because it is too closely related to the final label and may introduce target leakage.

Next, categorical variables were converted into numeric form. For example:

- “gender” was converted using one-hot encoding
- “stress_level” was converted using one-hot encoding
- “academic_work_impact” was converted into a binary numeric value

```
# Manual encoding for categorical columns
gender_map = {
    "Male": [1, 0, 0],
    "Female": [0, 1, 0],
    "Other": [0, 0, 1]
}

stress_map = {
    "Low": [1, 0, 0],
    "Medium": [0, 1, 0],
    "High": [0, 0, 1]
}

impact_map = {
    "No": [0],
    "Yes": [1]
}
```

After encoding, all features became numeric and could be used by the Perceptron.

Then, the data was normalized using standardization:

```

# =====
# 3. NORMALIZATION
# =====

# Normalize numeric range using training data only
feature_mean = X_train.mean(axis=0)
feature_std = X_train.std(axis=0)

# Avoid division by zero
feature_std[feature_std == 0] = 1.0

X_train = (X_train - feature_mean) / feature_std
X_test = (X_test - feature_mean) / feature_std

```

This process transforms each feature so that it has approximately:

- mean = 0
- standard deviation = 1

This step is necessary because the dataset contains features with very different scales. For example, “sleep_hours” may range from 4 to 9, while “notifications_per_day” may be much larger. If normalization is not applied, large-scale features may dominate the weight updates and make learning unstable or biased. Standardization helps the Perceptron learn more fairly from all features and improves the efficiency of training.

3-2. Model construction:

Define the Perceptron structure (weights W and bias b).

The model used in this assignment is a single-layer Perceptron for binary classification.

Its structure consists of:

- a weight vector W
- a bias term b

In code, the initialization was:

```
# =====
# 5. TRAINING PARAMETERS
# =====

epochs = 40
bias = 0.0
lr = 0.01
weights = np.zeros(X_train.shape[1])
```

If the number of input features is n , then:

- $W = [w_1, w_2, \dots, w_n]$
- b is a single scalar

For each input sample x , the Perceptron computes a linear combination:

$$z = W \cdot x + b$$

Then it applies an activation rule:

```
# =====
# 4. PERCEPTRON MODEL
# =====

def sign(z):
    return 1 if z >= 0 else 0
```

So the prediction is:

- 1 if $W \cdot x + b \geq 0$
- 0 if $W \cdot x + b < 0$

The full prediction function was:

```
# =====  
# 4. PERCEPTRON MODEL  
# =====  
  
def sign(z):  
    return 1 if z >= 0 else 0  
  
def perceptron(X, weights, bias):  
    scores = np.dot(X, weights) + bias  
    predictions = []  
    for score in scores:  
        predictions.append(sign(score))  
    return np.array(predictions, dtype=int)
```

This means the Perceptron learns a linear decision boundary that separates the two classes: addicted and not addicted.

3-3. Training process:

The Perceptron was trained using repeated passes through the training set over multiple epochs.

The main training logic was:

```

# =====
# 6. TRAINING
# =====

for epoch in range(epochs):
    updates_in_epoch = 0

    for x_i, y_i in zip(X_train, y_train):
        y_pred = perceptron(x_i.reshape(1, -1), weights, bias)[0]

        # Update only when wrong
        if y_i != y_pred:
            delta_w = lr * (y_i - y_pred) * x_i
            weights += delta_w

            delta_b = lr * (y_i - y_pred)
            bias += delta_b

```

The training process works as follows:

- Take one training sample “x_i” and its true label “y_i”
- Compute the predicted label “y_pred”
- Compare the prediction with the true label
- If the prediction is wrong, update the weights and bias

The weight update rule is:

- $W = W + lr * (y_i - y_{pred}) * x_i$
- $b = b + lr * (y_i - y_{pred})$

where:

- “lr” is the learning rate
- “y_i” is the true label
- “y_pred” is the predicted label
- “x_i” is the current input vector

This means:

- If the prediction is correct, no update is made
- If the prediction is wrong, the model adjusts the weights in the direction that helps correct the mistake

The idea behind this update is that the Perceptron strengthens or weakens the influence of input features depending on whether they contributed to a correct or incorrect classification. Over many training examples, these repeated adjustments gradually move the decision boundary to better separate the two classes.

In the implementation, the number of updates per epoch was also counted:

```
updates_in_epoch += 1
```

This was useful for convergence analysis, because if the number of updates becomes zero, it means the model has stopped changing its weights and has converged on the training data.

4. Discussion

4.1 Problem solving:

Yes, my model successfully classifies the given dataset. (But no convergence)

The approximate accuracy is:

- Train Accuracy: 0.8293
- Test Accuracy: 0.8300

```
=== Final Results ===
Trained Weights: [-0.04426438  0.18228178  0.12996286  0.01848295  0.03052954  0.01853795
-0.0090497  -0.0252591  -0.00504963 -0.02930054  0.03579377 -0.00634546
-0.00827302 -0.00498843  0.01321753  0.01960016]
Trained Bias: 0.09999999999999999
Train Acc: 0.8408
Test Acc: 0.8267

Number of correct test predictions: 1240
Number of wrong test predictions: 260
```

4.2 Convergence analysis:

The model never stops updating, even with 40 epochs.

```
Epoch 36/40 - Updates: 925 - Train Acc: 0.8433
Epoch 37/40 - Updates: 909 - Train Acc: 0.8467
Epoch 38/40 - Updates: 935 - Train Acc: 0.8372
Epoch 39/40 - Updates: 953 - Train Acc: 0.8597
Epoch 40/40 - Updates: 891 - Train Acc: 0.8408
```

The failure to converge suggests that the dataset is not linearly separable under the current feature representation. Since the perceptron is a single-layer linear classifier, it can only learn one linear decision boundary. The smartphone addiction data is based on human behavior variables such as screen time, gaming time, sleep, stress,

and academic/work impact, which likely contain overlap and noise between classes. Therefore, even after many epochs, some samples remain misclassified, causing continued updates.

4.3 Limitations:

Yes, I did encounter non-linear separability issues. As explained above, after reviewing the dataset, I found out that there are many overlapping features that could cause the non-linear separability issues.

4.4 Reflection:

I have learned a few new insights from doing this assignment:

First, weight adjustment in connectionism is a basic but very crucial method for machine learning. It enables the learning process for the models; learning in a connectionist model does not happen by storing explicit rules (as in a symbolic model), but by gradually adjusting connection weights based on errors in prediction. Each weight represents the strength of influence from an input feature to the output, so when the model makes a wrong classification, the update process shifts the decision boundary slightly. This helped me see that learning is essentially the accumulation of many small corrections. I also realized that weight adjustment is not always smooth or stable: when the data is not linearly separable, the Perceptron may continue updating without converging, which shows both the power and the limitation of simple (early) connectionist learning.

Another insight is that the weights are not just abstract numbers; they reflect how strongly different features contribute to the final decision. In the MNIST in-class exercise, the weights correspond to the importance of different pixel positions; while in the smartphone addiction task, they correspond to behavioral features such as screen time, sleep, stress, and app usage. This made the idea of connectionism more concrete for me: intelligence can emerge from many simple weighted connections rather than from symbolic logic alone.

The single-layer Perceptron plays an important historical and conceptual role in modern AI frameworks. Historically, it is one of the earliest neural network models and forms the foundation of later neural network development. Conceptually, it introduces the core ideas that are still central in modern AI: weighted inputs, bias, activation, prediction, error-driven updates, and iterative learning. Although a single-layer Perceptron is too limited for complex non-linear tasks, it remains important because it is the simplest useful example of how a neural model learns from data.

In modern AI, the single-layer Perceptron is no longer used for most of the advanced tasks by itself, but its core mechanism is still embedded inside deeper models. Multi-layer neural networks, deep learning systems, and transformer-based frameworks all build on the same basic principle of weighted connections and parameter adjustment. Therefore, the Perceptron serves as both a historical foundation and a conceptual building block for modern AI. By implementing it directly in this assignment, I was able to better understand how today's much more advanced models are rooted in the same foundational learning idea.